Material complementario · Lenguajes de programación



CONOCE EL LENGUAJE

C#

con .NET

Moderno, versátil y listo para la nube: un lenguaje, muchos destinos.

CURSO

Desarrollando con Inteligencia Artificial

INSTRUCTOR

Felipe Cuevas C.

ANTES DE EMPEZAR

Ficha técnica

Todo lo esencial de C# en una sola tarjeta. Si solo tienes un minuto, quédate con esta página.



LENGUAJE DE PROGRAMACIÓN

C# (se pronuncia "C sharp")

Familia de C · estándar ECMA-334 e ISO/IEC

Desde 2000

CREADOR **Anders Hejlsberg y su equipo en Microsoft**	MANTIENE HOY **Microsoft + comunidad de código abierto (.NET Foundation)**
PARADIGMAS **Orientado a objetos · Funcional · Genérico · Asíncrono · Orientado a eventos**	TIPADO **Estático y fuerte, con inferencia (`var`)**
EJECUCIÓN **Compilado a código intermedio (IL) y traducido en ejecución por el runtime**	MEMORIA **Automática con *garbage collector***
VERSIÓN ESTABLE (NOV. 2026) **C# 14 · .NET 10 LTS** Publicada el 11 de noviembre de 2025. C# 15 con .NET 11 está en fase final (RC) y se espera para noviembre de 2026	ARCHIVOS TÍPICOS `.cs` `.csproj` `.sln` `.razor` `.dll`

PLATAFORMAS: **Windows** · **Linux** · **macOS** · **iOS** · **Android** · **Navegador (WebAssembly)** · **Nube**

DATO CURIOSO

En música, "C♯" es la nota **do sostenido**: medio tono más alta que C. El nombre sugiere "un paso más allá de C". Su nombre clave interno fue **COOL** (*C-like Object Oriented Language*).

¿TE SUENA EL CREADOR?

Anders Hejlsberg también creó Turbo Pascal, fue el arquitecto de Delphi y, años después, de TypeScript.

1 INTRODUCCIÓN

De dónde viene

De apuesta corporativa de Microsoft a lenguaje libre, multiplataforma y premiado.

C# nació en **Microsoft** en el año 2000, diseñado por **Anders Hejlsberg**, el mismo arquitecto de Turbo Pascal y Delphi. Se creó junto con la plataforma **.NET** para unir la potencia de la familia C con la productividad y seguridad de un lenguaje moderno. En 2002 llegó su primera versión estable y se convirtió en el estándar del software empresarial en Windows. Desde 2016, con **.NET Core**, es libre, de código abierto y multiplataforma. Hoy corre en servidores, la nube, móviles e incluso en el navegador, y es el lenguaje de **Unity**, uno de los motores de videojuegos más usados del mundo.

HITO EN EL RANKING TIOBE

2× **Lenguaje del año**

Ganó el premio en 2023 y en 2025. En 2025 tuvo el mayor crecimiento anual de todo el ranking: +2,94 puntos.

¿Y en octubre de 2026?

Está en el **5.º lugar** del índice (3,79 %), detrás de Python, C, C++ y Java.

Línea de tiempo

2000 Microsoft presenta C# y .NET

2002 C# 1.0 y .NET Framework

2016 .NET Core: abierto y multiplataforma

2023 Primer premio TIOBE

2025 C# 14, .NET 10 y segundo premio

2026 C# 15 y .NET 11 en camino

> **Nació para Windows y terminó corriendo en todas partes.**
>
> Abrir su código en 2016 fue el giro que lo llevó a Linux, la nube y los contenedores.

ACLAREMOS ALGO CLAVE

C#, .NET y Visual Studio no son lo mismo

C# es el lenguaje. .NET es la plataforma que lo hace funcionar. Y el editor lo eliges tú.

CAPA 3 · DÓNDE ESCRIBES (IDE)

Visual Studio · VS Code · Rider

Editores con autocompletado, depurador y plantillas de proyecto. Puedes usar cualquiera.

CAPA 2 · LA PLATAFORMA

.NET (runtime + librerías + SDK)

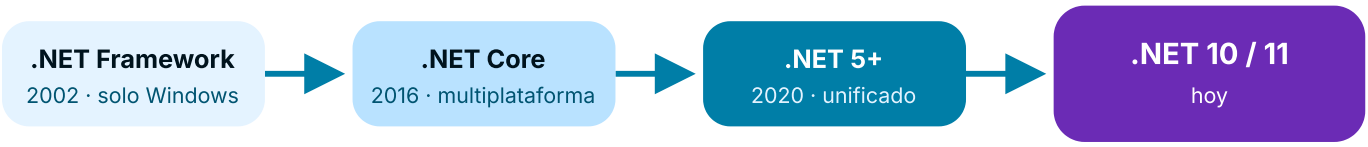
El runtime ejecuta tu programa; las librerías te dan miles de funciones listas; el SDK compila y publica.

CAPA 1 · EL IDIOMA

C#

Las reglas y palabras con que escribes: `class`, `var`, `async`, `record`...

La evolución de .NET



Las versiones pares (8, 10...) son LTS: tienen soporte por 3 años.

> **C# es el idioma; .NET es todo lo que lo hace funcionar.**
>
> Lenguaje, runtime, librerías y herramientas: juntos forman la plataforma.

¿POR QUÉ IMPORTA?

".NET Framework" es la versión antigua, solo para Windows. Cuando busques ayuda o pidas código a una IA, di **"C# 14 con .NET 10"** para evitar respuestas desactualizadas.

2 ESTRUCTURA

Cómo funciona

C# se compila dos veces: primero a un código intermedio y luego, al ejecutarse, a código de máquina.

¿Compilado o interpretado? Las dos cosas, en etapas

El compilador de C# (llamado **Roslyn**) traduce tu código a **IL** (*Intermediate Language*), un código intermedio guardado en archivos `.dll`. Cuando ejecutas el programa, el runtime de .NET (**CLR**) convierte ese IL en código de máquina "justo a tiempo" (**JIT**) y lo optimiza mientras corre.

MODO NORMAL (JIT)

📄 **Programa.cs**

▼ **compilador Roslyn**

📦 **Programa.dll (código IL)**

▼ **runtime CLR + JIT**

🖥️ **Código de máquina**

ALTERNATIVA: NATIVE AOT

📄 **Programa.cs**

▼ **compilación anticipada**

⚙️ **Ejecutable nativo**

▼ **sin JIT**

🖥️ **Arranque más rápido**

> **Escribes una vez y el runtime se encarga del resto.**
>
> El mismo .dll corre en Windows, Linux o macOS; el JIT lo adapta al procesador de cada equipo.

¿Y la memoria?

Garbage collector

Creas objetos con new y el *garbage collector* los elimina cuando ya nadie los usa. No necesitas liberarlos a mano.

Recursos externos

Archivos, conexiones y *sockets* sí se cierran explícitamente con `using`, que llama a `Dispose()` por ti.

ATENCIÓN A ESTO

El *garbage collector* libera memoria, pero **no cierra conexiones**. Olvidar el `using` con una base de datos puede agotar las conexiones disponibles.

2 ESTRUCTURA

Paradigmas, organización y versión actual

Paradigmas que soporta

Orientado a objetos
Clases, herencia, interfaces

Funcional
Lambdas, LINQ, `record`

Genérico
Tipos reutilizables `List<T>`

Asíncrono
`async` / `await`

C# popularizó el modelo **async/await**, que luego adoptaron JavaScript, Python y otros lenguajes.

Cómo se organiza el código: `namespace` y `class`

```
namespace Tienda;        // agrupa tipos

public class Calculadora
{
    // lo público: se usa desde afuera
    public int Sumar(int a, int b)
        => a + b;

    // lo privado: solo dentro de la clase
    private int Doble(int x)
        => x * 2;
}
```

namespace = la "carpeta" lógica que agrupa clases relacionadas.

public / private = la parte pública y la parte interna de cada clase.

Un proyecto se describe en un archivo `.csproj` y varios proyectos se agrupan en una solución.

Lo nuevo: una versión cada noviembre

C# 14 · nov. 2025 (actual)
Miembros de extensión, palabra clave `field` en propiedades y asignación con `?.`

.NET 10 · LTS
Versión con soporte de largo plazo (3 años): la recomendada para proyectos nuevos.

C# 15 · nov. 2026 (en RC)
Tipos unión, jerarquías cerradas y `break`/`continue` con etiqueta.

Quién lo mantiene
Microsoft, con desarrollo abierto en GitHub y aportes de la comunidad.

MANOS AL CÓDIGO

Primer contacto

Un programa completo y moderno en pocas líneas, siguiendo buenas prácticas.
Lee los números de la derecha mientras recorres el código.

```csharp
// Program.cs
var cursos = new List<Curso>        // ①
{
    new("C#", 30),
    new("Python", 24),
    new("Java", 40)
};

var largos = cursos                 // ②
    .Where(c => c.Horas >= 30)
    .Select(c => c.Nombre);

foreach (var nombre in largos)
{
    Console.WriteLine($"Curso largo: {nombre}"); // ③
}

record Curso(string Nombre, int Horas);  // ④
```

1 Sin ceremonia

No hace falta un método `Main`; `var` deduce el tipo por ti.

2 LINQ

Filtra y transforma colecciones con un estilo parecido a SQL.

3 Interpolación

El `$` permite insertar variables dentro del texto con `{ }`.

4 record

Una clase de datos completa en una sola línea.

SALIDA

```
Curso largo: C#
Curso largo: Java
```

> **Sin `Free`, sin punteros: el runtime cuida la memoria.**
>
> Tú te concentras en la lógica; el *garbage collector* limpia cuando ya no usas los objetos.

3 PRINCIPALES USOS

Dónde brilla

Es uno de los lenguajes más versátiles: del servidor al videojuego, pasando por la nube y el móvil.

Backend y APIs
Servicios web de alto rendimiento con ASP.NET Core.

Nube
Microservicios y contenedores en Azure, AWS o Google Cloud.

Finanzas y empresas
Banca, seguros, ERP y sistemas internos de grandes compañías.

Videojuegos
Es el lenguaje de scripting del motor Unity.

Escritorio
Apps de Windows con WPF y WinForms.

Móvil multiplataforma
Apps para Android, iOS y escritorio con .NET MAUI.

Casos reales

Stack Overflow
Comunidad de desarrolladores

El sitio de preguntas y respuestas más conocido del mundo dev funciona con .NET y la nube.

Fidelity
Servicios financieros · EE. UU.

Construyó su plataforma Active Trader Pro con .NET MAUI para más de **50 millones** de clientes.

Microsoft
Teams, Bing y Copilot

Servicios de Teams y Bing migraron a .NET moderno, y el equipo de Copilot rehízo su backend con .NET.

Y EN EL COMERCIO ELECTRÓNICO

nopCommerce, una plataforma de tiendas en línea hecha en ASP.NET Core, impulsa más de 60 mil tiendas. Todos estos casos están publicados por Microsoft en su página de clientes de .NET.

4 PARA QUÉ SE RECOMIENDA

¿Cuándo elegirlo?

Ningún lenguaje sirve para todo. Este semáforo te ayuda a decidir con criterio.

APIs y microservicios	🟢 **Ideal**	ASP.NET Core está entre los frameworks web más rápidos y trae inyección de dependencias, seguridad y documentación de APIs integradas.
Nube y contenedores	🟢 **Ideal**	Imágenes livianas, Native AOT para arranque rápido y Aspire para orquestar servicios. Funciona en cualquier nube.
Apps empresariales	🟢 **Ideal**	Tipado fuerte, herramientas maduras y soporte de largo plazo: justo lo que piden bancos y grandes empresas.
Videojuegos	🟢 **Ideal**	Unity usa C# como lenguaje principal.
Escritorio Windows	🟢 **Ideal**	WPF y WinForms son estándares de la industria para apps de Windows.
Integrar IA en una app	🟢 **Muy bueno**	SDK oficial de Anthropic, Microsoft Agent Framework y SDK oficial de MCP en C#.
Móvil multiplataforma	🟢 **Muy bueno**	.NET MAUI comparte código entre Android, iOS, macOS y Windows.
Web frontend	🟡 **Posible**	Blazor permite hacer interfaces web en C#, aunque JavaScript y TypeScript siguen dominando el navegador.
Machine learning	🟡 **Posible**	ML.NET entrena modelos clásicos, pero la investigación y los modelos grandes viven en Python.
Big data y bajo nivel	🔴 **No recomendado**	Spark y Hadoop son del mundo JVM y Python; drivers y sistemas embebidos son terreno de C, C++ o Rust.

> **Elige C# cuando necesites un backend robusto que crezca contigo.**
>
> Es una opción segura para APIs, nube y software empresarial; para ML o big data, combínalo con Python.

5 ECOSISTEMA

Tu caja de herramientas

.NET trae mucho “de fábrica” y NuGet suma miles de paquetes de la comunidad. Así se organiza.

Web y APIs

ASP.NET Core · El framework web principal: APIs, MVC y Minimal APIs.

Blazor · Interfaces web interactivas escritas en C#.

Datos

Entity Framework Core · ORM oficial: trabaja con la base de datos usando objetos y LINQ.

Dapper · Micro-ORM liviano y muy rápido para SQL directo.

Nube y microservicios

Aspire · Orquesta y monitorea apps distribuidas en tu equipo y en la nube.

gRPC para .NET · Comunicación rápida entre microservicios.

Inteligencia artificial

Microsoft Agent Framework · Agentes y flujos con IA (sucesor de Semantic Kernel).

SDK de Anthropic y SDK de MCP · Conectar con Claude y crear servidores MCP.

ML.NET · Machine learning clásico dentro de .NET.

Apps y juegos

.NET MAUI · Apps móviles y de escritorio multiplataforma.

WPF y WinForms · Apps de escritorio para Windows.

Unity · Motor de videojuegos programado en C#.

Calidad y paquetes

xUnit y NUnit · Frameworks de pruebas unitarias.

NuGet · El gestor de paquetes oficial de .NET.

TAMAÑO DEL ECOSISTEMA

Es **grande y muy completo**: Microsoft respalda las piezas principales (web, datos, nube, IA) y la comunidad aporta el resto a través de NuGet. Rara vez tendrás que buscar fuera de .NET.

CONEXIÓN CON EL CURSO

C# + IA

Dos caminos: usar la IA para **programar** en C# y usar C# para **construir apps con IA**.

La IA te ayuda a programar

C# tiene muchísimo código público, así que asistentes como Claude lo manejan muy bien: generan APIs, escriben pruebas y ayudan a migrar proyectos antiguos.

C# construye apps con IA

El **SDK oficial de Anthropic** (paquete NuGet `Anthropic`) conecta tu app con Claude. Con **Microsoft Agent Framework** creas agentes y con el **SDK de MCP** expones herramientas.

LIMITACIÓN REAL

Como C# existe hace más de 20 años, la IA a veces mezcla **.NET Framework con .NET moderno**, usa patrones antiguos (como `Startup.cs`) o sintaxis de versiones viejas. Revisa, compila y prueba siempre.

Buenas prácticas al pedir código C#

Indica la versión

"C# 14 con .NET 10", no solo "C#".

Di el tipo de proyecto

Minimal API, MVC, Blazor, MAUI o consola.

Pide código seguro

`async/await`, tipos que admiten nulos activados y `using`.

Comparte contexto

Pega el `.csproj` y las clases relacionadas.

Prompt de ejemplo para copiar

```
// Prompt
Actúa como desarrollador senior de C# 14 con .NET 10.
Crea una Minimal API para gestionar Cursos (listar, crear, eliminar)
con Entity Framework Core y SQLite. Usa async/await y tipos que admiten
nulos activados. Agrega pruebas con xUnit y explícame cada archivo
en una lista de 5 puntos. No uses Startup.cs ni APIs de .NET Framework.
```

Flujo sugerido con IA

1 Especificar — Describe qué debe hacer y sus reglas.

2 Generar — Pide el código en pasos pequeños.

3 Probar — Corre las pruebas con `dotnet test`.

4 Revisar — Lee el código y entiende cada línea.

6 RESUMEN

Para recordar

C# es un lenguaje moderno, de tipado fuerte y multiparadigma, creado por Microsoft y hoy libre y multiplataforma gracias a **.NET**. Se compila a código intermedio que el runtime optimiza al ejecutar, y la memoria la gestiona el *garbage collector*. Brilla en APIs, nube, software empresarial, escritorio y videojuegos con Unity. Con una versión nueva cada año, es una apuesta segura para tu carrera.

Versátil
Web, nube, escritorio, móvil y juegos con un solo lenguaje.

Gestionado
El runtime y el *garbage collector* cuidan la memoria.

Vigente
Nueva versión cada noviembre y LTS cada dos años.

BIBLIOGRAFÍA

Fuentes consultadas · consultadas el 5 de noviembre de 2026

OFICIALES · MICROSOFT

- Microsoft Learn (2026). *What's new in .NET 11*. learn.microsoft.com/dotnet/core/whats-new/dotnet-11/overview
- Microsoft Learn. *What's new in C# 14* y *C# language documentation*. learn.microsoft.com/dotnet/csharp
- Microsoft. *Your success, powered by .NET* (casos de clientes). dotnet.microsoft.com/platform/customers
- Microsoft. *.NET and .NET Core Support Policy*. dotnet.microsoft.com

IA Y SDK

- Anthropic. *C# SDK*. platform.claude.com/docs · nuget.org/packages/Anthropic
- Microsoft. *Microsoft Agent Framework*. learn.microsoft.com

RANKING Y PRENSA

- TIOBE Software (oct. 2026). *TIOBE Index*. tiobe.com/tiobe-index
- ADTmag (07-01-2026). *Tiobe Language Rankings for 2025*. adtmag.com
- Visual Studio Magazine (08-01-2026). *C# Grabs Language of the Year*. visualstudiomagazine.com
- InfoQ (feb. 2026). *.NET 11 Preview 1 Arrives*. infoq.com

HISTORIA

- Wikipedia. *C Sharp (programming language)* y *.NET*. en.wikipedia.org

PARA SEGUIR APRENDIENDO

- Documentación de C#: learn.microsoft.com/dotnet/csharp
- Descarga de .NET: dotnet.microsoft.com/download



MATERIAL ELABORADO POR

Felipe Cuevas C.

Instructor y autor del curso **Desarrollando con Inteligencia Artificial**. Ingeniero informático, desarrollador senior y docente universitario.

felipecuevas.dev

Material complementario para estudiantes · noviembre de 2026